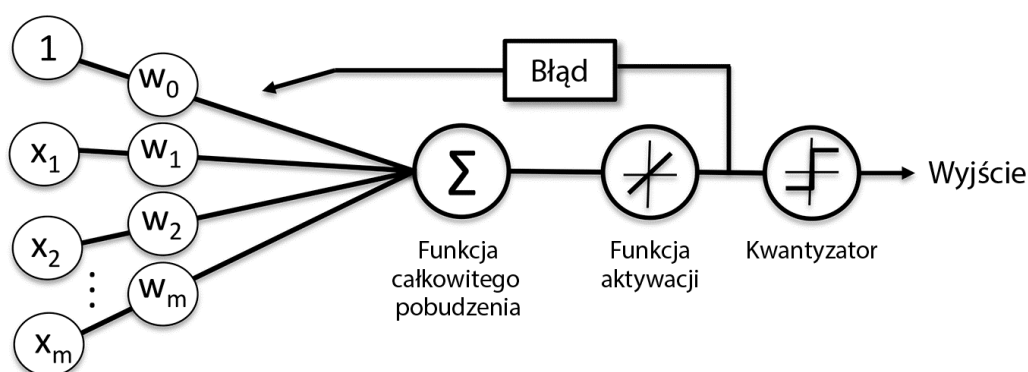


Adaptacyjne neurony liniowe i zbieżność uczenia

W tym podrozdziale przyjrzymy się kolejnej odmianie jednowarstwowej sieci neuronowej: **ADaptacyjnemu Liniowemu NEuronowi (ADALINE)**. Model Adaline został zaprezentowany zaledwie kilka lat po algorytmie perceptronu Rosenblatta przez Bernarda Widrowa i jego doktora Tedda Hoffa; adaptacyjny neuron liniowy można uznać za twórcze rozwinięcie koncepcji perceptronu (B. Widrow i in., *An adaptive „Adaline” neuron using chemical „memistors”*, „Number Technical Report” 1553-2, Stanford Electron. Labs, Stanford, California, October 1960). Algorytm Adaline jest szczególnie interesujący, gdyż zaprezentowana w nim została kluczowa koncepcja definiowania i minimalizowania funkcji kosztu, co stanowi podstawę zrozumienia bardziej zaawansowanych algorytmów klasyfikujących, takich jak regresja logistyczna czy maszyny wektorów nośnych, a także modeli regresji omówionych w dalszych rozdziałach.

Podstawową różnicą pomiędzy regułą uczenia Adaline (zwaną również **regułą Widrowa-Hoffa**) a perceptronem Rosenblatta jest sposób traktowania wag: w przypadku adaptacyjnego neuronu liniowego wagi są aktualizowane na podstawie liniowej funkcji aktywacji, a nie funkcji skoku jednostkowego (jak to ma miejsce w perceptronie). W modelu Adaline taka liniowa funkcja aktywacji $\phi(z)$ jest po prostu funkcją tożsamościową całkowitego pobudzenia w taki sposób, że $\phi(w^T x) = w^T x$.

Liniowa funkcja aktywacji służy do obliczania wag, natomiast podobny do funkcji skoku jednostkowego **kwantyzator** jest stosowany do przewidywania etykiet klas, co zostało zaprezentowane na rysunku 2.9.



Rysunek 2.9. Schemat modelu Adaline

Jeżeli porównamy rysunek 2.9 ze schematem algorytmu perceptronu (rysunek 2.4), zauważymy, że różnica polega na stosowaniu wartości ciągłych obliczanych za pomocą liniowej funkcji aktywacji do wyliczania błędów modelu i aktualizowania wag, a nie binarnych etykiet klas.

Minimalizacja funkcji kosztu za pomocą metody gradientu prostego

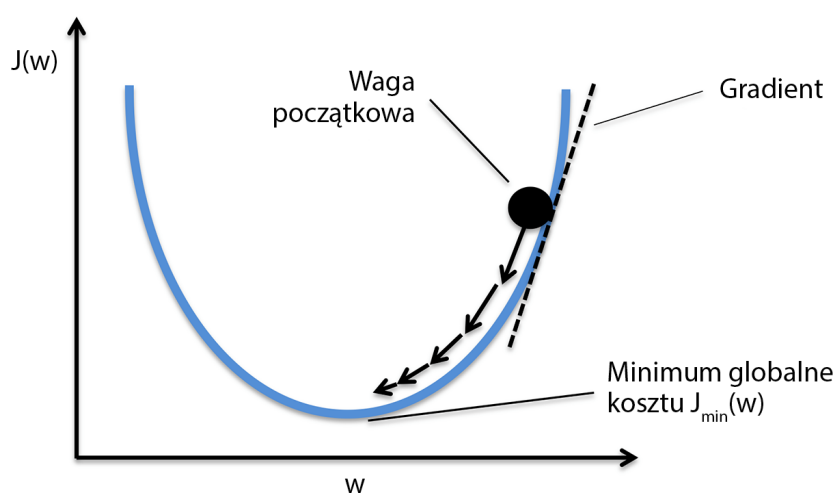
Jednym z najistotniejszych zadań w algorytmach nadzorowanego uczenia maszynowego jest zdefiniowanie **funkcji celu**, która będzie optymalizowana w procesie nauki. Funkcja celu często przyjmuje postać **funkcji kosztu**, którą pragniemy zminimalizować. W przypadku modelu Adaline możemy wyznaczyć funkcję kosztu J , wyznaczającą wagi za pomocą **sumy kwadratów błędów** (ang. *Sum of Squared Errors* — **SSE**) pomiędzy wyliczonymi wynikami a rzeczywistymi etykietami klas:

$$J(w) = \frac{1}{2} \sum_i (y^{(i)} - \phi(z^{(i)}))^2$$

Wartość $\frac{1}{2}$ została dodana jedynie dla naszej wygody; łatwiej nam będzie w ten sposób wyprowadzić gradient, o czym przekonamy się w dalszej części rozdziału. Główną zaletą tej liniowej funkcji aktywacji jest — w przeciwieństwie do funkcji skoku jednostkowego — umożliwienie

różniczkowania funkcji kosztu. Inną przydatną własnością jest wypukłość funkcji kosztu; pozwala nam ją wykorzystywać prosty, ale potężny algorytm optymalizacyjny, zwany **gradientem prostym** (ang. *gradient descent*); umożliwia on znajdowanie wag minimalizujących funkcję kosztu klasyfikującą próbki zawarte w zbiorze danych Iris.

Na rysunku 2.10 widzimy, że działanie algorytmu gradientu prostego możemy przyrównać do **schodzenia z góry** aż do osiągnięcia lokalnego lub globalnego minimum kosztu. W każdej iteracji schodzimy coraz niżej, a rozmiar kolejnego kroku jest określany przez wartości współczynnika uczenia, jak również przez nachylenie gradientu.



Rysunek 2.10. Schemat poglądowy działania algorytmu gradientu prostego

Za pomocą gradientu prostego możemy zaktualizować wagi poprzez oddalenie się od gradientu $\nabla J(\mathbf{w})$ naszej funkcji kosztu $J(\mathbf{w})$:

$$\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}$$

Zmiana wagi $\Delta \mathbf{w}$ jest tu zdefiniowana jako ujemny gradient pomnożony przez współczynnik uczenia η :

$$\Delta \mathbf{w} = -\eta \nabla J(\mathbf{w})$$

Aby wyliczyć gradient funkcji kosztu, musimy obliczyć pochodną cząstkową tej funkcji przy uwzględnieniu każdej wagi w_j , $w_j, \frac{\partial J}{\partial w_j} = -\sum_i (y^{(i)} - \phi(z^{(i)})) x_j^{(i)}$, dzięki czemu będziemy mogli zapisać aktualizację wagi w_j jako $\Delta w_j = -\eta \frac{\partial J}{\partial w_j} = \eta \sum_i (y^{(i)} - \phi(z^{(i)})) x_j^{(i)}$. Aktualizujemy jednocześnie wszystkie wagi, dlatego reguła uczenia Adaline przyjmuje postać $\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}$.

Osobom zaznajomionym z aparatem matematycznym przedstawiam sposób wyprowadzenia pochodnej cząstkowej funkcji kosztu za pomocą sumy kwadratów błędów w odniesieniu do j-tej wagi:

$$\begin{aligned}
 \frac{\partial J}{\partial w_j} &= \frac{\partial}{\partial w_j} \frac{1}{2} \sum_i (y^{(i)} - \phi(z^{(i)}))^2 = \\
 &= \frac{1}{2} \frac{\partial}{\partial w_j} \sum_i (y^{(i)} - \phi(z^{(i)}))^2 = \\
 &= \frac{1}{2} \sum_i 2(y^{(i)} - \phi(z^{(i)})) \frac{\partial}{\partial w_j} (y^{(i)} - \phi(z^{(i)})) = \\
 &= \sum_i (y^{(i)} - \phi(z^{(i)})) \frac{\partial}{\partial w_j} \left(y^{(i)} - \sum_i (w_j^{(i)} x_j^{(i)}) \right) = \\
 &= \sum_i (y^{(i)} - \phi(z^{(i)})) (-x_j^{(i)}) = \\
 &= -\sum_i (y^{(i)} - \phi(z^{(i)})) x_j^{(i)}
 \end{aligned}$$

Mimo że reguła uczenia w modelu Adaline wygląda identycznie jak w przypadku perceptronu, wartość funkcji $\phi(z^{(i)})$, gdzie $z^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)}$, stanowi liczbę rzeczywistą, a nie liczbę całkowitą etykiety klas. Ponadto aktualizacja wag jest obliczana na podstawie wszystkich próbek z zbioru uczącym (czyli wagi nie są przyrostowo aktualizowane po sprawdzeniu każdej próbki), dlatego właśnie ta technika bywa również nazywana metodą wsadową gradientu prostego (ang. *batch gradient descent*).